ImGee: Avionics & Flight-Sim UI Toolkit

Comprehensive Developer & User Manual • ImGee v1.0.0

FlyWithLua 2.8.20+ -- ImGee Modern UI Suite -- Dear ImGui 1.92.8

Comprehensive Reference for Markdown, TimedTooltip, Tactile Widgets, Sized Typography, and Dynamic Scaling

November 2026 Author: *Steven L. Goldberg (`@slgoldberg`) and the Rogers* ***Target Release:*** *FlyWithLua 2.8.20+ (Built against Dear ImGui 1.92.8 & ImGee Suite v1.0.0 / ImgWindow v2.4.0)*

---

1. Overview & Architecture

FlyWithLua integrates **LuiGee** (*"The Lua Companion to ImGee"*), a modern flight-deck UI toolkit, directly into the Lua scripting environment. This brings tactile avionics widgets, intelligent stationary tooltips, CommonMark 4.1 Markdown, sized typography, and zero-disk texture services to X-Plane floating windows.

All extension bindings are implemented natively in C++ (`src/FloatingWindows/FWLImGeeBindings.cpp`), exposing zero-overhead wrappers that bridge Lua scripts to `ImgWindow`'s header-only `ImGee` suite.

Key Capabilities

- **Semantic Markdown Typography:** Render rich Markdown text with headings, bold, italics, monospace code blocks, unordered and numbered lists, and clickable hyperlinks.
- **Visual Pills, Badges & Custom Colors:** Inline `<pill>`, `<badge>`, `<kbd>`, and `<color #rrggbb>` formatting for cockpit annunciators, system status, and flight telemetry.
- **Intelligent Stationary Tooltips (`TimedTooltip`):** Stationary hover tooltips that do not jitter or track the mouse, feature reading-adaptive dwell timeouts, enforce layout style isolation, and fully support embedded Markdown.
- **Tactile UX Widgets:** Smooth animated `ToggleButton`, segmented `ToggleButtons`, `SliderWithReset` (with reset icon button), `PercentSlider`, mouse-wheel scroll chasers, and hyperlink widgets.
- **Universal C++ & PascalCase Alias Parity:** Strict parity with C++ Dear ImGui casing (`imgui.Markdown.Text`, `imgui.TimedTooltip.Text`, `imgui.ToggleButton`), accompanied by short aliases (`imgui.MD.Text`, `MD.Text`, `TimedTooltip.Text`) and case-tolerant fallbacks.
- **Variadic String Formatting:** Pass `printf`-style format specifiers (`%s`, `%d`, `%.2f`) directly into any text or tooltip function without needing manual calls to `string.format()`.

---

2. Markdown Extension (`imgui.Markdown` / `MD`)

The Markdown extension parses standard Markdown syntax and renders it as native Dear ImGui draw commands within the current window.

Semantic Font Auto-Wiring

The Markdown system automatically hooks into FlyWithLua's **Named Font Registry**:

Markdown Element	Target Font Name	Fallback Font	Visual Style
Heading 1 (`#`)	`Roboto-Medium-20`	`Roboto-Medium`, Default	20px Bold / Underlined
Heading 2 (`##`)	`Roboto-Medium-16`	`Roboto-Medium`, Default	16px Bold / Underlined
Heading 3 (`###`)	`Roboto-Medium-16`	`Roboto-Medium`, Default	16px Bold

Markdown Element	Target Font Name	Fallback Font	Visual Style
Bold (`**bold**`)	`Roboto-Medium`	`Roboto-Regular`, Default	Medium / Semi-Bold weight
Italics (`*italic*`)	`Roboto-Italic-16`	`Roboto-Italic`, `Roboto-Light`	True Italic / Slanted weight
Monospace / Code (`code`)	`RobotoMono-Regular-16`	`RobotoMono-Regular`, `ProggyForever`	Scalable Monospace font

(Note: True italic and scalable monospace typographies are powered by `Roboto-Italic.ttf` and `RobotoMono-Regular.ttf`, bundled directly in `FlyWithLua/Custom_Fonts/`. If not present, the engine gracefully falls back to `Roboto-Light` or `ProggyForever`).

Markdown Functions

`imgui.Markdown.Text(format_string, ...)`

Renders formatted Markdown text. Supports variadic arguments.

```
[LUA]
imgui.Markdown.Text("# Pre-Flight Inspection")
imgui.Markdown.Text("Verify engine oil pressure: **%.1f PSI** (Nominal: %d-%d PSI)", oil_psi, 45, 60)
```

`imgui.Markdown.TextWrapped(format_string, ...)`

Renders formatted Markdown text with automatic line wrapping to the current window boundary.

```
[LUA]
imgui.Markdown.TextWrapped("This is a long flight briefing paragraph that wraps smoothly across the window...")
```

`imgui.Markdown.TextUnformatted(string)`

Renders Markdown text directly without variadic format parsing (safe for strings containing arbitrary `%` characters).

```
[LUA]
imgui.Markdown.TextUnformatted("Direct text containing 100% pure data.")
```

`imgui.Markdown.ButtonMD(label, [tooltip], [wrap_width])`

Renders an interactive push button supporting procedural vector icons, shortcodes, Unicode glyphs, hand cursor on hover, and rich Markdown tooltips.

- **Procedural Vector Icons:** Automatically renders vector geometry directly on the button face via XML tags (`<icon caret-up />`), shortcodes (`:undo:`), raw Unicode (▲, ↺), or link macros (`#[^](ICON_CARET_UP)`).
- **Auto-Centering:** Centers the icon and text dynamically, maintaining crisp layout on any font atlas.

```
[LUA]
-- Vector icon + text button:
if imgui.Markdown.ButtonMD("Quick Actions <icon caret-up />##btn", "**Quick Actions**\n\nFlight deck snippets me
    toggle_quick_actions()
end

-- Pure icon button:
if imgui.Markdown.ButtonMD("<icon caret-up />##scroll_top", "Scroll to top") then
    scroll_top()
end
```

Dedicated Vector Icon Buttons

- **`imgui.IconButton(str_id, icon_name, [w, h], [tooltip]) -> clicked`**: Pure vector icon button auto-sized to a crisp square.
- **`imgui.ButtonIcon(label, icon_name, [w, h], [icon_on_right], [tooltip]) -> clicked`**: Button with both procedural vector icon and text.

```lua
[LUA]
if imgui.IconButton("##up_arrow", "caret-up", 0, 0, "Move Note Up") then
    move_note_up()
end

if imgui.ButtonIcon("Settings", "gear", 0, 0, false, "Configure Cockpit Notes") then
    open_settings()
end
```

`width, height = imgui.Markdown.CalcTextSize(text, [wrap_width])`

Calculates the pixel dimensions required to render a given Markdown string before drawing.

```lua
[LUA]
local w, h = imgui.Markdown.CalcTextSize("### Heading Example")
```

---

3. Inline Styling: Pills, Badges, Keys & Colors

FlyWithLua supports inline XML-style tags embedded directly inside Markdown text:

1. Status Pills (`<pill color>TEXT</pill>`)

Pills render rounded, solid-colored badges with contrasting text -- ideal for system annunciators and warning flags.

Supported color keywords: **`green`**, **`red`**, **`blue`**, **`yellow`**, **`orange`**, **`purple`**, **`gray`**, **`error`**, **`warning`**, **`caution`**, **`info`**, **`success`**.

Pills and badges also natively support embedded icons via **`icon=`**, **`img=`**, or **`image=`**, as well as self-closing unary annunciators (**`<pill icon=... />`**):

```lua
[LUA]
imgui.MD.Text("Hydraulics: <pill green>SYSTEM NORMAL</pill> | Fuel Tank A: <pill warning>LOW LEVEL</pill>")
-- Embedded icon pill:
imgui.MD.Text("Autopilot: <pill icon=avionics_beacon color=#3498db>NAV / ALT</pill>")
-- Self-closing unary annunciator:
imgui.MD.Text("Master Caution: <pill icon=avionics_beacon color=#e74c3c />")
```

2. Version & Topic Badges (`<badge color>TEXT</badge>`)

Badges render slightly more subtle categorized chips for metadata, build tags, and feature markers.

```lua
[LUA]
imgui.MD.Text("FlyWithLua <badge blue>v2.8.20</badge> running on <badge purple>Metal / Vulkan</badge>")
imgui.MD.Text("Firmware: <badge icon=avionics_beacon>v2.3.0</badge>")
```

3. Keyboard Shortcut Tags (`<kbd>Key</kbd>`)

Renders a bordered, keycap-styled box for visual keyboard shortcuts.

```lua
[LUA]
imgui.MD.Text("Press <kbd>Shift</kbd> + <kbd>Ctrl</kbd> + <kbd>R</kbd> to reload Lua scripts.")
```

4. Custom Hex Colors & Alpha Transparency (`#rgb`, `#rgba`, `#rrggbb`, `#rrggbbaa`)

Colors arbitrary text spans, pills, badges, or style definitions using standard web hexadecimal notation with optional alpha transparency:

- `#rgb` (12-bit RGB, opaque)
- `#rgba` (16-bit RGBA with 4-bit alpha)
- `#rrggbb` (24-bit RGB, opaque)
- `#rrggbbaa` (32-bit RGBA with full 8-bit alpha)

```
[LUA]
imgui.MD.Text("Target speed: <color #00bc8c>250 knots</color> below <color #f39c12>10,000 ft</color>.")
imgui.MD.Text("Translucent overlay: <pill #3498db88>TRANSLUCENT PILL</pill>")
```

Smart Capsule Auto-Contrast & Decoupled Backgrounds

- **Automatic Contrast:** When passing a bare color keyword or hex code into `<pill>` or `<badge>` (e.g. `<pill #ff4444>`), ImGee calculates the background color luminance and automatically renders the enclosed text in crisp, readable high-contrast black or white.
- **Decoupled Text & Background:** When specifying an explicit `bg=` or `bgcolor=` (e.g. `<style fg=#ffffff bg=#88888840>`), the text foreground color is strictly preserved and decoupled from the background fill box.
- **Recursive Tag Nesting:** Style tags can be nested recursively within other tags (e.g. `<style green bold>Exterior: <style yellow>WARN</style> Clear</style>`).

5. Declarative Semantic Style Tags & Macro Aliases

Rather than repetitively writing verbose formatting strings or nesting raw XML tags across scripts, developers can define custom semantic Markdown tags dynamically during script initialization:

1. Semantic Style Tags (`imgui.RegisterStyleTag`)

Maps a custom tag name to a composite typography style combining colors, weights, fonts, and background capsules:

```
[LUA]
-- Define semantic tags during script startup:
imgui.RegisterStyleTag("cam", "green bold mono")
imgui.RegisterStyleTag("viewmode", "mono bg=#88888840")
imgui.RegisterStyleTag("alert", "bg=#ff4444 fg=#ffffff bold")

-- Use seamlessly inside any Markdown or StyledText:
imgui.MD.Text("Camera Axis: <cam>roll</cam> | View: <viewmode>CHASE_CAM</viewmode>")
imgui.MD.Text("<alert>PULL UP</alert> Terrain ahead!")
```

2. Declarative Tag Aliases (`imgui.RegisterTagAlias`)

Expands any arbitrary Markdown or XML template string. Any occurrence of `%s` inside the template is dynamically replaced with the enclosed inner text:

```
[LUA]
-- Register macro template aliases:
imgui.RegisterTagAlias("tip", "<style yellow><b>Tip: </b></style> %s")
imgui.RegisterTagAlias("axis", "**Axis: <style cyan bold>%s</style>**")

-- Stamp anywhere in Markdown:
imgui.MD.Text("<tip>Check altimeter setting before descent.</tip>")
imgui.MD.Text("<axis>ELEVATOR</axis> trim centered.")
```

3. Smart Tag Registration (`imgui.RegisterTag`)

A flexible unified registration entry point that automatically detects the handler type:

- **Function:** Registers an interactive Lua callback: `function(inner_text, param) ... end`
- **String with `%s`:** Automatically registers as a template alias (`RegisterTagAlias`)
- **String without `%s`:** Automatically registers as a semantic style tag (`RegisterStyleTag`)

```lua
-- Interactive Lua callback tag:
imgui.RegisterTag("metric", function(inner_text, param)
    imgui.TextStyled("<pill #3498db>%s</pill> <small color=#aaaaaa>%s</small>", inner_text, param ~= "" and para
end)

-- Used in Markdown:
imgui.MD.Text("Field Elevation: <metric ft>5,420</metric>")
```

6. Multi-Line Fenced Code Blocks (`lang`)

Standard CommonMark triple backticks (or tildes `~~~`) render multi-line code blocks in shaded charcoal containers with rounded corners and an automatic language indicator chip in the upper corner:

````
[MARKDOWN]

```lua
-- Autopilot vertical speed calculation
local target_vvi = (target_alt - current_alt) * 0.1
if target_vvi > 2000 then target_vvi = 2000 end
set("sim/cockpit2/autopilot/vvi_dial_fpm", target_vvi)
```
````

7. Clickable Hyperlinks & Images

Standard CommonMark Markdown links and images are supported out of the box:

- **Clickable Links:** `[Label](URL)`
- **Images:** `![Alt Text](path/to/image)`

When a hyperlink is clicked, FlyWithLua automatically invokes the native operating system browser or document viewer (via `open` on macOS, `ShellExecute` on Windows, or `xdg-open` on Linux).

```lua
[LUA]

imgui.MD.Text("For aircraft manuals, see the [FlyWithLua Documentation](https://github.com/sparker256/FlyWithLua
```

4. TimedTooltip Extension (`imgui.TimedTooltip` / `TimedTooltip`)

The standard Dear ImGui tooltip (`imgui.SetTooltip`) suffers from several UX limitations in desktop flight simulators:

- **Cursor tracking / Jitter:** The tooltip position moves with every sub-pixel mouse movement, distracting the pilot.
- **Zero Reading Intelligence:** Disappears instantly when the mouse nudges off the target, regardless of how much text remains to read.
- **Style Contamination:** Inherits whatever window padding, spacing, or color overrides happen to be active on the widget stack.

`TimedTooltip` completely resolves these issues through a dedicated tooltip controller:

1. **Stationary Anchor:** Anchors at the position where hover was initiated and remains rock-solid until dismissed.
2. **Reading-Adaptive Timeout:** Dwell duration scales dynamically based on the word count of the tooltip content.
3. **Style Isolation:** Rendered in an independent style context with clean standard padding and borders.

4. **Rich Markdown Formatting:** Supports full headings, bold, italics, pills, badges, and colors via `TimedTooltip.MD.Text`.

TimedTooltip Functions

`imgui.TimedTooltip.Text(format_string, ...)`

Shows a stationary plain text tooltip when the preceding widget is hovered.

```lua
imgui.Button("AP Master##btn")
imgui.TimedTooltip.Text("Flight Director and Autopilot Master Engagement Switch")
```

`imgui.TimedTooltip.MD.Text(format_string, ...)`

Shows a stationary tooltip rendered with full Markdown and pill styling.

```lua
imgui.Button("Pitot Heat##btn")
imgui.TimedTooltip.MD.Text("### Pitot Tube Heating System\n\n" ..
                           "Status: <pill green>ON</pill>\n\n" ..
                           "* Prevents ice accumulation in pitot static probes.\n" ..
                           "* <color #f39c12>Caution: Ground operation limited to 5 minutes.</color>")
```

Dwell & Dimension Controls

Configure tooltip behavior globally or per session:

- **`imgui.TimedTooltip.SetHoverDelay(cycles)`** -- Number of frames cursor must hover before tooltip appears.
- **`imgui.TimedTooltip.SetMaxDisplay(cycles)`** -- Base display timeout in frames.
- **`imgui.TimedTooltip.SetMaxWidth(pixels)`** -- Maximum wrap width in pixels (defaults to 400px).
- **`imgui.TimedTooltip.SetMinWidth(pixels)`** -- Minimum width in pixels.
- **`imgui.TimedTooltip.Clear()`** -- Immediately dismisses any active tooltip.

---

5. Modern LuiGee Widgets

The `ImGee` widgets suite (`imgui.widgets.*` / `imgui.*`) adds tactile, specialized controls for simulation settings and telemetry adjustment.

`changed, new_val = imgui.ToggleButton(label, bool_val)`

An animated iOS-style sliding pill toggle switch with smooth tactile color transitions.

```lua
local changed, new_state = imgui.ToggleButton("Cabin Lights", cabin_lights_on)
if changed then
    cabin_lights_on = new_state
end
```

`changed, new_state = imgui.ToggleButtons(label, int_state, off_label, on_label)`

A segmented multi-state toggle control.

```lua
local changed, new_mode = imgui.ToggleButtons("##RecogLights", light_mode, "OFF", "STROBE")
if changed then
    light_mode = new_mode
end
```

`changed, new_val = imgui.SliderWithReset(label, val, min, max, default_val)`

A horizontal slider paired with an adjacent reset button (↺) that instantly returns the slider to its calibrated default value.

```lua
local changed, new_trim = imgui.SliderWithReset("Aileron Trim", aileron_trim, -1.0, 1.0, 0.0)
if changed then
    aileron_trim = new_trim
end
```

`changed, new_val = imgui.PercentSlider(label, val)`

A slider designed for normalized floating point values `[0.0 .. 1.0]`, automatically displayed and formatted as an integer percentage (`0% .. 100%`).

```lua
local changed, new_flaps = imgui.PercentSlider("Flaps Deflection", flaps_deploy)
if changed then
    flaps_deploy = new_flaps
end
```

`changed, new_val = imgui.PercentSliderWithReset(label, val, default_val)`

Combines the normalized percent slider with an automatic reset button.

```lua
local changed, new_mix = imgui.PercentSliderWithReset("Mixture", fuel_mixture, 1.0)
if changed then
    fuel_mixture = new_mix
end
```

`changed, new_val = imgui.AdjustItemOnMouseWheel(val, min, max, [step])`

Detects mouse wheel scrolling when hovering over the preceding item (e.g. a button or custom area) and increments or decrements the floating-point value.

```lua
imgui.Button(string.format("NAV 1: %.2f MHz", nav1_freq), 220, 30)
local changed, new_freq = imgui.AdjustItemOnMouseWheel(nav1_freq, 108.00, 117.95, 0.05)
if changed then
    nav1_freq = new_freq
end
```

`changed, new_idx = imgui.CycleItemOnMouseWheel(idx, count, [wrap])`

Cycles an integer index when the mouse wheel is scrolled over the preceding widget. Supports standard 1-based Lua indexing.

```lua
local gear_modes = { "UP", "OFF", "DOWN" }
imgui.Button(string.format("Gear Lever: %s", gear_modes[gear_idx]), 200, 30)
local changed, new_idx = imgui.CycleItemOnMouseWheel(gear_idx, #gear_modes, true)
if changed then
    gear_idx = new_idx
end
```

`clicked = imgui.ButtonLink(label, [url])` and `imgui.TextLink(label, [url])`

Renders a button or text item styled as a hyperlink. If `url` is supplied, clicking automatically opens the URL in the system browser.

```lua
imgui.TextLink("Check METAR Weather Reports", "https://aviationweather.gov/metar")
imgui.ButtonLink("FlyWithLua Forum Support", "https://forums.x-plane.org")
```

`imgui.Spinner(label, [radius], [thickness], [color], [speed])`

Renders an organic, sinusoidal expanding-and-contracting activity arc for loading or asynchronous tasks. Procedurally rendered via `ImDrawList` antialiased paths with zero textures and zero font dependencies.

- `label`: Unique ID or optional text label rendered beside the spinner.
- `radius`: Outer radius in pixels (defaults to `0.55 * GetFontSize()`).
- `thickness`: Arc stroke width (defaults to `radius * 0.22`, min 2px).
- `color`: RGBA color table `{r, g, b, [a]}` or 32-bit packed color integer (defaults to theme checkmark / button active).
- `speed`: Animation speed multiplier (defaults to `1.0`).

```lua
-- Standard themed spinner
imgui.Spinner("Loading Flight Plan...")

-- Custom styled aviation-blue spinner at 1.2x speed
imgui.Spinner("FMS Ingestion##fms", 16, 3.0, {0.2, 0.7, 1.0, 1.0}, 1.2)
```

`imgui.RadialTimer(label, fraction, [radius], [thickness], [color], [bg_color], [clockwise])`

Renders a circular countdown dial representing progress or elapsed time (`0.0 .. 1.0`).

- Sweeps from the 12 o'clock position (`-90°`).
- Supports custom arc color and background ring track color.
- `clockwise`: `true` for clockwise sweep (default), `false` for counter-clockwise.

```lua
-- 65% complete fuel burn cycle dial
imgui.RadialTimer("Fuel Burn Dial", 0.65, 18, 3.5, {0.2, 0.85, 0.3, 1.0})
```

`imgui.CountdownTimer(label, current_sec, max_sec, [radius], [thickness], [color], [bg_color], [clockwise])`

Convenience helper wrapping `RadialTimer` with remaining seconds and total max seconds.

```lua
-- 42 seconds remaining of a 60 second approach timer
imgui.CountdownTimer("Approach Timer (42s / 60s)", 42.0, 60.0, 20, 4.0, {1.0, 0.65, 0.0, 1.0})
```

`clicked = imgui.ResetButton(str_id, [is_modified], [draw_frame])`

A standalone DPI-scaled vector undo/reset button (↺) matching standard ImgWindow widget sizing.

- `is_modified`: `true` renders the glyph in standard white/active styling. `false` (baseline) renders it dimmed/disabled.
- `draw_frame`: Whether to draw a button bounding frame or leave it flat/transparent.

```lua
[LUA]
local reset_clicked = imgui.ResetButton("##reset_zoom", current_zoom ~= 100, false)
if reset_clicked then
    current_zoom = 100
end
```

`clicked, is_copied = imgui.CopyButton(str_id, text_to_copy)`

Renders a 1-click clipboard copy button with automatic transient visual feedback (changing label to `Copied!` in green for 1.5 seconds) and built-in `TimedTooltip` integration.

- Also aliased as `imgui.CopyBufferButton`.

```lua
[LUA]
imgui.CopyButton("##copy_notes", editor_text)
```

`clicked, is_copied = imgui.CopyLastItemButton(str_id, [prefix], [suffix])`

A non-intrusive chaser copy button that attaches to the preceding widget on the same line (such as an `InputText` or label) and copies its content with optional prefix and suffix formatting.

- Also aliased as `imgui.CopyTextFieldButton`.

```lua
[LUA]
imgui.InputText("##airport_code", icao_buf, 8)
imgui.SameLine()
imgui.CopyLastItemButton("##copy_icao")
```

`start_idx, end_idx, cur_pos = imgui.GetInputTextSelection()`

Inspects the active Dear ImGui text input or multiline buffer.

- Returns 0-based character indices `[start_idx, end_idx)` and current cursor position `cur_pos`.
- Returns `-1, -1, -1` when no input text item is active or no selection is made.

`imgui.SetInputTextSelection(start_idx, end_idx)`

Programmatically sets or restores the active selection range in the active text input box and automatically reactivates the input widget so keyboard focus is retained.

```lua
[LUA]
-- Wrap selected text in bold asterisks
local s, e = imgui.GetInputTextSelection()
if s >= 0 and e > s then
    editor_text = editor_text:sub(1, s) .. "**" .. editor_text:sub(s + 1, e) .. "**" .. editor_text:sub(e + 1)
    imgui.SetInputTextSelection(s + 2, e + 2)
end
```

`delta = imgui.GetMouseWheel([consume])` and `imgui.GetMouseWheelEither([consume])`

Low-level mouse wheel query primitives.

- `imgui.GetMouseWheel(consume)`: Returns vertical scroll wheel clicks.

- **`imgui.GetMouseWheelEither(consume)`**: Returns vertical scroll clicks, falling back to horizontal scroll clicks (ideal for Mac trackpads and horizontal mice).
- When **`consume`** is **`true`**, immediately consumes the event by zeroing **`io.MouseWheel`** so underlying parent windows do not scroll inadvertently.

Standalone Procedural Badges, Pills & Keycaps

While badges and pills are frequently used inline inside Markdown strings, FlyWithLua also exposes them as standalone procedural widgets:

- **`imgui.Badge(text, [variant])`** -- Subtle categorization chip.
- **`imgui.Pill(text, [variant])`** -- Solid-colored rounded pill.
- **`imgui.Kbd(text)`** -- Framed keyboard keycap.

Variants: **`"default"`**, **`"primary"`**, **`"success"`** (**`"green"`**), **`"warning"`** (**`"yellow"`**), **`"danger"`** (**`"red"`**), **`"info"`** (**`"blue"`**).

```lua
imgui.Badge("METAR OK", "success")
imgui.SameLine()
imgui.Pill("AUTOPILOT ENGAGED", "primary")
imgui.SameLine()
imgui.Kbd("Ctrl+Shift+P")
```

In-Cockpit File Dialogs & File Picker Suite

FlyWithLua integrates turnkey C++11 file selection and directory scanning widgets (**`imgui_file_dialog.h`**) built directly into ImgWindow.

- **Microsecond Native Scanning:** Uses POSIX **`opendir`** on macOS/Linux and Win32 **`FindFirstFileW`** on Windows -- completely eliminating Windows **`io.popen("dir /b ...")`** console window flashes, cmd.exe deadlocks, and subprocess overhead.
- **In-Viewport & VR-Safe:** Renders modals inside the simulator viewport; never stalls the X-Plane graphics thread, and never loses window focus.

`changed, chosen_file = imgui.FilePickerCombo(label, current_file, directory, [extensions], [flags])`

An inline toolbar dropdown that scans the target directory and lists matching files for fast switching.

```lua
local changed, chosen_file = imgui.FilePickerCombo("##notes", current_note, "FlyWithLua/Scripts/CockpitNotes/not
if changed then
    current_note = chosen_file
    load_note_file(current_note)
end
```

`confirmed, chosen_path, still_open = imgui.OpenFileDialog(title, is_open, [starting_dir], [extensions])`

A modal, in-cockpit file browser with breadcrumb navigation (**`[ < Back ]`**, **`[ Up ]`**), interactive live search filtering, and human-readable file sizes (**`KB`**, **`MB`**). Double-clicking a file confirms immediately.

```lua
local confirmed, chosen_path, still_open = imgui.OpenFileDialog("Select Flight Plan", is_dialog_open, "Output/FM
if confirmed then
    load_flight_plan(chosen_path)
end
is_dialog_open = still_open
```

`confirmed, save_path, still_open = imgui.SaveFileDialog(title, is_open, [starting_dir], [default_filename], [extensions])`

A modal in-cockpit file saver with filename text input and automatic extension enforcement (if the user omits `.md`, the extension is automatically appended).

```[LUA]
local confirmed, save_path, still_open = imgui.SaveFileDialog("Save Notes As", is_save_open, "FlyWithLua/Scripts
if confirmed then
    save_note_file(save_path)
end
is_save_open = still_open
```

6. Dynamic ImGui Mouse Cursors

FlyWithLua now natively enables **dynamic software ImGui mouse cursors by default** inside floating windows:

- **Interactive Hand (`ImGuiMouseCursor_Hand`):** Shows automatically when hovering over links, buttons, reset glyphs, and toggle switches.
- **Text I-Beam (`ImGuiMouseCursor_TextInput`):** Appears when hovering over editable text inputs.
- **Resize Cursors (`ImGuiMouseCursor_ResizeEW`, `ResizeNS`, etc.):** Seamlessly transition when dragging columns, tables, or frames.
- **Edge Resize Passthrough:** As the mouse approaches the outer boundary of an in-sim window, ImGui automatically releases cursor control to X-Plane so native hardware resize handles function without hitching.

Disabling or Customizing Cursors

If a user or script author prefers standard OS hardware cursor rendering, dynamic cursors can be toggled at multiple levels:

1. **Global Preference (`fwl_prefs.ini`):**

```[INI]
; Set to 0 to disable dynamic ImGui cursors globally
ImguiMouseCursors = 0
```

2. **Per-Window Lua Control:**

```[LUA]
-- Disable custom cursors on this window:
float_wnd_set_imgui_cursor_handling(wnd, false)

-- Query cursor state:
local is_active = float_wnd_get_imgui_cursor_handling(wnd)
```

3. **ImGui Namespace Control:**

```[LUA]
imgui.SetCursorHandling(false) -- Disable globally
imgui.SetCursorHandling(wnd, false) -- Disable for specific window
```

7. Directional Popups, Pulldowns & Inline StyledText (Phase 3)

FlyWithLua includes Phase 3 interactive UX extensions from **`ImgWindow v2.3`**:

1. `imgui.SelectPulldown(label, current_item, items, [tooltips], [width], [anchor], [align], [wrap], [natural_list_order]) -> changed, new_item`

Renders a compact, single-line dataref / options dropdown selector with a clean, procedural antialiased vector caret (or custom icon font glyph).

- **Procedural Vector Carets:** When no icon font is defined, renders a crisp vector triangle via `ImDrawList` that scales perfectly and never suffers from missing-glyph Unicode diamonds.
- **Directional Blossoming (`anchor`):** Supports `imgui.PopupAnchor.BelowItem` (default down-pointing) or `imgui.PopupAnchor.AboveItem` (blossoms UP from bottom docks/status bars with an up-pointing caret!). When open, carets automatically flip orientation.
- **Scroll Pass-Through When Closed:** Mouse-wheel events pass directly through to the parent container, so scrollable windows scroll smoothly without mutating settings.
- **Dynamic Wheel Cycling When Open:** Spinning the mouse wheel while the popup is open cycles through items dynamically without closing the popup!
- **"Lens UI" Open-Menu Tooltip Latching:** When spinning the mouse wheel inside an open popup, per-item tooltips dynamically target and lock onto the newly focused item until the user deliberately moves the mouse.
- **Wrap & Natural Direction (`wrap`, `natural_list_order`):** Configurable circular wrapping across index boundaries (`wrap = true`) and natural wheel stepping direction (`natural_list_order = true`).
- **Timed Tooltips Support:** Accepts an optional table of per-item hover tooltips (using smooth `TimedTooltip` to eliminate cursor jitter).
- **Markdown Shorthand Aliases:** Also available as `imgui.SelectPulldownMD`, `imgui.Markdown.SelectPulldown`, and `imgui.MD.SelectPulldown`.
- **0-Based Indices:** `current_item` and the returned `new_item` are 0-based integers.

```lua
[LUA]

local nav_sources = { "GPS 1 (WAAS)", "VOR 1 (114.70)", "LOC / ILS", "FMS 1" }
local nav_tips    = { "Garmin GPS 1 primary", "Oakland VOR", "Instrument Landing System", "Flight Management Sys
local active_nav  = 0 -- 0-based

-- Downward dropdown selector with circular wrapping and natural list order:
local changed, new_idx = imgui.SelectPulldown("##NavSel", active_nav, nav_sources, nav_tips, 280,
    imgui.PopupAnchor.BelowItem, imgui.PopupAlign.Left, true, true)
if changed then
    active_nav = new_idx
end

-- Upward blossoming selector (for bottom status bars / toolbars):
local changed_dock, new_dock_idx = imgui.SelectPulldown("##DockSel", active_nav, nav_sources, nav_tips, 200,
    imgui.PopupAnchor.AboveItem, imgui.PopupAlign.Right)
```

Global Caret Configuration

- `imgui.SetDefaultPulldownCaret(down_caret, [up_caret])`: Configure global icon glyphs (e.g. `ICON_FA_CHEVRON_DOWN`, `ICON_FA_CHEVRON_UP`) or pass `nil` to use procedural vector arrows.

2. Directional Popup Item Anchoring

Anchors popup windows relative to the preceding item's bounding box (`GetItemRectMin` / `Max`) with sub-pixel precision.

- **Directional Anchors (`imgui.PopupAnchor`):**
- `imgui.PopupAnchor.AboveItem` (1): Blossoms **UP** above the item (indispensable for status bars or toolbars docked along the screen bottom).
- `imgui.PopupAnchor.BelowItem` (0): Standard downward dropdown menu.
- `imgui.PopupAnchor.Auto` (2): Automatically chooses Above or Below based on available screen space.
- **Alignment Modes (`imgui.PopupAlign`):**

- **`imgui.PopupAlign.Left`** (0): Align left edge with item left edge.
- **`imgui.PopupAlign.Right`** (1): Align right edge with item right edge.
- **`imgui.PopupAlign.Center`** (2): Center horizontally over item.
- **`imgui.PopupAlign.Auto`** (3): Automatically detect boundary edges.
- **Muscle-Memory Toggle Safety:** Clicking the opener button while the popup is open cleanly toggles it closed without re-opening flicker.
- **Headroom Reporting:** **`imgui.GetLastPopupHeight(str_id)`** and **`imgui.GetLastPopupSize(str_id)`** query the exact measured dimensions of the popup window for host window expansion without guessing.

```lua
[LUA]
-- Opening anchored popup above a button in a bottom toolbar:
if imgui.Button("View Options ▲##btnView") then
    imgui.OpenPopupAtItem("##ViewMenu", imgui.PopupAnchor.AboveItem, imgui.PopupAlign.Left)
end

if imgui.BeginAnchoredPopup("##ViewMenu") then
    imgui.TextDisabled("CAMERA MODES")
    imgui.Separator()
    if imgui.MenuItem("Cockpit 3D") then SetView(0) end
    if imgui.MenuItem("Spot View")   then SetView(1) end
    imgui.Separator()
    if imgui.MenuItem("Close") then imgui.CloseCurrentPopup() end
    imgui.EndPopup()
end

-- Query popup dimensions:
local width, height = imgui.GetLastPopupSize("##ViewMenu")
```

3. Standalone Inline StyledText (**`imgui.TextStyled`** / **`imgui.StyledText`**)

Renders lightweight, inline rich text with **continuous baseline alignment** without requiring full Markdown document parsing. Also natively recognized inside full Markdown documents and scratchpads (**`CockpitNotes`**):

- **Full Formatted Variadic `printf` Support:**

All **`TextStyled`** and **`TextStyledWrapped`** variants accept a format string followed by variadic arguments. In Lua, this is automatically dispatched to the native **`string.format(...)`** engine, allowing seamless dynamic value formatting:

```lua
[LUA]
  imgui.TextStyled("Altitude: <huge>%.0f</huge> <small>%s</small> | Speed: <mono>%.1f</mono> <small>KIAS</small>
```

- **Supported Inline Tags:**
- **`<b>bold</b>`**, **`<strong>bold</strong>`**
- **`<i>italic</i>`**, **`<em>italic</em>`**
- **`<mono>monospace</mono>`**, **`<code>code</code>`**
- **`<h1>`** .. **`<h3>`** inline headings
- **`<small>reduced text</small>`** (Slot 2, typically 12px)
- **`<tiny>micro text</tiny>`** (Slot 2 fallback or dedicated 10px)
- **`<big>larger text</big>`** or **`<large>...</large>`** (Slot 1, typically 16px)
- **`<huge>hero text</huge>`** (Slot 4, 16px+ bold)
- **`<regular>reset to body</regular>`** or **`<sans>...</sans>`** (Slot 0, 14px)
- **`<color name>`** or **`<color #rgb|#rgba|#rrggbb|#rrggbbaa>`** (all size tags also accept optional **`color=`**, e.g. **`<tiny color=#888>v1.0.2</tiny>`**)
- **`<pill color>`** and **`<badge color>`** tags with optional **`icon=`** attributes and auto-contrast text

- **Functions:**
- `imgui.TextStyled(format_string, ...)` / `imgui.StyledText(...)`
- `imgui.TextStyledWrapped(format_string, ...)` / `imgui.StyledTextWrapped(...)`
- `imgui.TextStyledUnformatted(string)` / `imgui.StyledTextUnformatted(...)`

```lua
[LUA]
imgui.TextStyled("Mode: <b><color #2ecc71>CRUISE</color></b> | Mach: <mono>%.2f</mono>", 0.82)
imgui.TextStyled("Firmware: <tiny color=#888888>v1.0.2</tiny> | Status: <pill green>ACTIVE</pill>")
imgui.TextStyled("Heading: <huge>FL340</huge> <small color=gray>(Optimal)</small>")
imgui.TextStyledWrapped("<i>This paragraph</i> wraps automatically across the column width using the native StyledText wrapping")
```

4. Semantic Font Role Direct Text Helpers

Direct immediate-mode text rendering helpers mapped to the typography hierarchy:

- `imgui.SmallText("Caption text: %s", status)` / `imgui.SmallTextUnformatted(str)`
- `imgui.TinyText("Footnote: %s", rev)` / `imgui.TinyTextUnformatted(str)`
- `imgui.TitleText("Section Header: %s", title)` / `imgui.TitleTextUnformatted(str)`
- `imgui.MonoText("Dataref: %s", dref_name)` / `imgui.MonoTextUnformatted(str)`

5. Semantic Font Roles (`imgui.PushFontRole` / `imgui.PopFontRole`)

Pushes an explicit typography role from the semantic registry:

- Roles: `imgui.FontRole.Regular`, `Title`, `Small`, `Bold`, `BoldLarge`, `Monospace`, `Italic`, `MonospaceMedium`, `Heading1`, `Heading2`, `Heading3`.

```lua
[LUA]
imgui.PushFontRole(imgui.FontRole.Monospace)
imgui.TextUnformatted("COM1: 121.500 MHz")
imgui.PopFontRole()
```

6. First-Class Named Badges & Pills (`imgui.BadgeNamed` / `imgui.PillNamed`)

Directly render pills and badges using named textures/icons:

```lua
[LUA]
imgui.PillNamed("avionics_beacon", "Mode C", {0.18, 0.80, 0.44, 1.0})
imgui.BadgeNamed("avionics_beacon", "v2.3.0")
```

7. In-Memory Image Service (`imgui.RegisterMemoryImage`)

Decode and register raw image files (PNG, JPEG, etc.) directly from memory byte strings into GPU VRAM with **zero disk I/O, zero file permissions issues, and zero temp files**:

- `success, width, height = imgui.RegisterMemoryImage(name, binary_string_data)`
- `imgui.UnregisterMemoryImage(name)` / `imgui.DeleteMemoryImage(name)`
- `exists, width, height = imgui.GetNamedTexture(name)` / `imgui.HasMemoryImage(name)`
- `imgui.ImageNamed(name, [width, height])`

Once registered via `imgui.RegisterMemoryImage("my_chart", png_bytes)`:

- **Pure ImGui Lua:** `imgui.ImageNamed("my_chart", 300, 200)`
- **Markdown Text:** `![Approach Chart](my_chart)`
- **Inline StyledText:** `<img name="my_chart" width=300 height=200 />`

- **Pills & Badges:** `<pill icon=my_chart color=#3498db>CHART</pill>` or `imgui.PillNamed("my_chart", "CHART")`

```lua
-- Register an image downloaded via LuaSocket HTTP or embedded via Base64:
local ok, w, h = imgui.RegisterMemoryImage("weather_radar", raw_radar_png_bytes)
if ok then
    imgui.ImageNamed("weather_radar", w, h)
end
```

8. Universal C++ & Lua Alias Matrix

FlyWithLua provides comprehensive alias coverage matching both standard Dear ImGui PascalCase conventions and C++ namespace aliases:

Category	Primary API	Short Alias	Global Scope	Suffix Style
Markdown Text	`imgui.Markdown.Text`	`imgui.MD.Text`	`Markdown.Text` / `MD.Text`	`imgui.TextMD`
Markdown Wrapped	`imgui.Markdown.TextWrapped`	`imgui.MD.TextWrapped`	`Markdown.TextWrapped`	`imgui.TextWrappedMD`
Markdown Direct	`imgui.Markdown.TextUnformatted`	`imgui.MD.TextUnformatted`	`Markdown.TextUnformatted`	`imgui.TextUnformattedMD`
Markdown Button	`imgui.Markdown.ButtonMD`	`imgui.MD.ButtonMD`	`Markdown.ButtonMD`	`imgui.ButtonMD`
TimedTooltip Plain	`imgui.TimedTooltip.Text`	`imgui.TimedTooltip.text`	`TimedTooltip.Text`	--
TimedTooltip MD	`imgui.TimedTooltip.MD.Text`	`imgui.TimedTooltip.TextMD`	`TimedTooltip.MD.Text`	`TimedTooltip.TextMD`
Toggle Button	`imgui.ToggleButton`	`imgui.toggle_button`	--	--
Segmented Toggles	`imgui.ToggleButtons`	`imgui.toggle_buttons`	--	--
Slider With Reset	`imgui.SliderWithReset`	`imgui.slider_with_reset`	--	`imgui.ResettableSlider`
Percent Slider	`imgui.PercentSlider`	`imgui.percent_slider`	--	`imgui.SliderPercent`
Percent Slider w/ Reset	`imgui.PercentSliderWithReset`	`imgui.percent_slider_with_reset`	--	`imgui.ResettablePercentSlider`
Reset / Undo Button	`imgui.ResetButton`	`imgui.reset_button`	--	--
Mouse Wheel Adjust	`imgui.AdjustItemOnMouseWheel`	`imgui.adjust_item_on_mouse_wheel`	--	--
Mouse Wheel Cycle	`imgui.CycleItemOnMouseWheel`	`imgui.cycle_item_on_mouse_wheel`	--	--
Core Mouse Wheel	`imgui.GetMouseWheel`	`imgui.get_mouse_wheel`	--	`imgui.GetMouseWheelEither`
Activity Spinner	`imgui.Spinner`	`imgui.spinner`	--	`imgui.ActivitySpinner`
Radial Timer Dial	`imgui.RadialTimer`	`imgui.radial_timer`	--	`imgui.ProgressDial`
Countdown Timer	`imgui.CountdownTimer`	`imgui.countdown_timer`	--	`imgui.CountdownDial`
Copy Button	`imgui.CopyButton`	`imgui.copy_button`	--	`imgui.CopyBufferButton`

Category	Primary API	Short Alias	Global Scope	Suffix Style
Copy Last Item	`imgui.CopyLastItemButton`	`imgui.copy_last_item_button`	--	`imgui.CopyTextFieldButton`
Selection Inspect	`imgui.GetInputTextSelection`	`imgui.get_input_text_selection`	--	--
Selection Set	`imgui.SetInputTextSelection`	`imgui.set_input_text_selection`	--	--
Badge Widget	`imgui.Badge`	`imgui.badge`	--	`imgui.Tag`
Pill Widget	`imgui.Pill`	`imgui.pill`	--	--
Named Badge	`imgui.BadgeNamed`	`imgui.badge_named`	--	--
Named Pill	`imgui.PillNamed`	`imgui.pill_named`	--	--
Keycap Widget	`imgui.Kbd`	`imgui.kbd`	--	`imgui.Key` / `imgui.Keycap`
Hyperlinks	`imgui.ButtonLink`	`imgui.button_link`	--	`imgui.TextLink`
Icon Button	`imgui.IconButton`	`imgui.icon_button`	--	--
Button With Icon	`imgui.ButtonIcon`	`imgui.button_icon`	--	--
Mouse Cursor State	`imgui.SetCursorHandling`	`imgui.set_cursor_handling`	--	`imgui.EnableMouseCursors`
File Picker Combo	`imgui.FilePickerCombo`	`imgui.file_picker_combo`	--	`imgui.FilePicker`
Open File Dialog	`imgui.OpenFileDialog`	`imgui.open_file_dialog`	--	`imgui.OpenDialog`
Save File Dialog	`imgui.SaveFileDialog`	`imgui.save_file_dialog`	--	`imgui.SaveDialog`
Select Pulldown	`imgui.SelectPulldown`	`imgui.select_pulldown`	`imgui.SelectPulldownMD`	`imgui.MD.SelectPulldown`
Set Pulldown Caret	`imgui.SetDefaultPuldownCaret`	`imgui.set_default_pulldown_caret`	--	--
Open Popup At Item	`imgui.OpenPopupAtItem`	`imgui.open_popup_at_item`	--	--
Open Popup At Pos	`imgui.OpenPopupAtPos`	`imgui.open_popup_at_pos`	--	--
Begin Anchored Popup	`imgui.BeginAnchoredPopup`	`imgui.begin_anchored_popup`	--	--
Get Last Popup Height	`imgui.GetLastPopupHeight`	`imgui.get_last_popup_height`	--	--
Get Last Popup Size	`imgui.GetLastPopupSize`	`imgui.get_last_popup_size`	--	--
Styled Text	`imgui.TextStyled`	`imgui.StyledText`	--	`imgui.text_styled`
Styled Text Wrapped	`imgui.TextStyledWrapped`	`imgui.StyledTextWrapped`	--	`imgui.text_styled_wrapped`
Small Text	`imgui.SmallText`	`imgui.small_text`	`imgui.SmallTextUnformatted`	--
Tiny Text	`imgui.TinyText`	`imgui.tiny_text`	`imgui.TinyTextUnformatted`	--
Title Text	`imgui.TitleText`	`imgui.title_text`	`imgui.TitleTextUnformatted`	--
Mono Text	`imgui.MonoText`	`imgui.mono_text`	`imgui.MonoTextUnformatted`	--

Category	Primary API	Short Alias	Global Scope	Suffix Style
Font Role Push	`imgui.PushFontRole`	`imgui.push_font_role`	--	--
Font Role Pop	`imgui.PopFontRole`	`imgui.pop_font_role`	--	--
Memory Image Register	`imgui.RegisterMemoryImage`	`imgui.register_memory_image`	--	--
Memory Image Delete	`imgui.UnregisterMemoryImage`	`imgui.DeleteMemoryImage`	--	`imgui.delete_memory_image`
Memory Image Query	`imgui.GetNamedTexture`	`imgui.HasMemoryImage`	--	`imgui.has_memory_image`
Image Render by Name	`imgui.ImageNamed`	`imgui.image_named`	--	--
Register Style Tag	`imgui.RegisterStyleTag`	`imgui.register_style_tag`	`imgui.Markdown.RegisterStyleTag`	`imgui.RegisterMarkdownStyleTag`
Register Tag Alias	`imgui.RegisterTagAlias`	`imgui.register_tag_alias`	`imgui.Markdown.RegisterTagAlias`	`imgui.RegisterMarkdownTagAlias`
Register Tag (Smart)	`imgui.RegisterTag`	`imgui.register_tag`	`imgui.Markdown.RegisterTag`	`imgui.RegisterMarkdownTag`

9. Complete Interactive Script Recipe

Here is a minimal, complete script showing all extensions working together:

[LUA]

```
if not SUPPORTS_FLOATING_WINDOWS then return end

local wnd = float_wnd_create(540, 420, 1, true)
float_wnd_set_title(wnd, "Avionics LuiGee Demo")
float_wnd_set_imgui_builder(wnd, "build_avionics_wnd")

local ap_master = true
local altitude_target = 10000

function build_avionics_wnd(wnd, x, y)
    -- Markdown Header with Pill
    MD.Text("# Flight Management System <badge blue>ACTIVE</badge>")
    MD.Text("Current status: <pill green>EN ROUTE</pill> | Baro: **29.92 inHg**")
    imgui.Separator()

    -- Animated Toggle with TimedTooltip
    local changed, new_ap = imgui.ToggleButton("Autopilot Master", ap_master)
    if changed then ap_master = new_ap end
    TimedTooltip.MD.Text("### Autopilot Master Control\n" ..
                         "Toggles coupled lateral and vertical flight directors.\n" ..

                         "<color #00bc8c>Safe to disengage during cruise flight.</color>")

    -- Altitude Target with Mouse Wheel Tuning
    imgui.Spacing()
    imgui.Button(string.format("Target Altitude: %5d ft (Scroll Wheel)##alt", altitude_target), 300, 30)
    local alt_changed, new_alt = imgui.AdjustItemOnMouseWheel(altitude_target, 0, 45000, 500)
    if alt_changed then altitude_target = math.floor(new_alt) end
    TimedTooltip.Text("Scroll mouse wheel over button to adjust target altitude in 500 ft steps.")

    -- Hyperlink to Documentation
    imgui.Spacing()
    imgui.Separator()
    imgui.TextLink("FlyWithLua Online Documentation", "https://github.com/sparker256/FlyWithLua")
end
```

10. Bundled Demonstration Scripts & Add-ons

FlyWithLua includes production-grade example scripts and turnkey add-ons pre-installed in **`FlyWithLua/Scripts/`**:

1. `ImGui_LuiGee_Demo.lua` (Visual Feature Showcase)

Pre-installed in **`FlyWithLua/Scripts/`** and registered with the menu macro **`Plugins > FlyWithLua > FlyWithLua Macros > FlyWithLua: LuiGee Modern UI Demo`** and command **`FlyWithLua/LuiGee/toggle`**. It remains dormant until opened by the user.

A comprehensive interactive sandbox window demonstrating:

- Headings, soft-wrapping, bold, italic, and strikethrough typography.
- Nested multi-line code blocks with syntax highlighting, copy buttons, and CommonMark indent conformance.
- Tactile avionics widgets: animated toggles, segmented toggles, resettable sliders, percent sliders, and 270° rotary dials.
- Procedural spinners, sweep dials, and countdown timers.
- Interactive TimedTooltips with rich embedded Markdown.
- **Dynamic Proportional Window Zoom:** 50% to 200% canvas zoom with automatic window aspect-ratio scaling, 1-point exact quantum steppers, and 1-click A/B toggle memory.
- **Phase 3 Widgets & Sized Typography:** Zero-scroll-leak **`SelectPulldown`** with in-menu wheel cycling, directional popups blossoming UP or DOWN (**`OpenPopupAtItem`**), standalone inline **`TextStyled`** typography with embedded icons, inline size tags (**`<tiny>`**, **`<small>`**, **`<regular>`**, **`<big>`**, **`<huge>`**), direct semantic text helpers (**`imgui.SmallText`**, **`imgui.TinyText`**, **`imgui.TitleText`**, **`imgui.MonoText`**), and zero-disk in-memory GPU texture decoding (**`imgui.RegisterMemoryImage`**).

2. `CockpitNotes` Add-on (`CockpitNotes.lua`)

A full-featured, professional in-cockpit notes and briefing pad located in **`FlyWithLua/Scripts/CockpitNotes/`**:

- **Three Workflow Modes:** Fullscreen **Write**, side-by-side **Split**, and distraction-free flight **Preview**.
- **Tactile Canvas Font Zoom:** Range from 50% to 200% with exact 1-point quantum steppers, mouse wheel adjust, and 1-click A/B toggle back to 100% baseline.
- **Dedicated Multi-Note Storage:** Automatic file persistence in **`FlyWithLua/Scripts/CockpitNotes/notes/`** with active note file selection, manual Finder file discovery, and instant new note creation.
- **Flight Operations Checklists:** Rapid task insertion and formatting using verify-and-execute **`- [ ]`** syntax.
- **Aviation Quick Templates:** Pre-configured templates for ATIS copy pads, IFR CRAFT clearances, pre-flight checklists, and flight diversion briefings.
- **About Dialog:** Live Markdown-rendered documentation and credits.